

DOCUMENTO DE REQUISITOS DEL PRODUCTO (PRD) - FRONT-END

Proyecto: Tienda Online Tipo "Amazon" (Desarrollo Guiado por IA)

Componente: Interfaz de Usuario (Front-end)

Metodología: Extreme Programming (XP) / Desarrollo Agéntico

1. Contexto Metodológico y de Inteligencia Artificial

1.1. Integración de Metodología Ágil (XP)

El desarrollo del front-end se estructurará bajo los principios de **Extreme Programming (XP)**, priorizando el desarrollo iterativo, los lanzamientos pequeños y el diseño simple. Dado que el código será generado por herramientas de IA, el ciclo de XP se adaptará mediante "Prompts de Programación en Pareja" (*Pair Programming con IA*), donde el desarrollador actúa como el rol de *Driver/Reviewer* (revisor de arquitectura y validador de calidad) y los agentes actúan como el *Navigator/Coder* (generadores de código y pruebas).

1.2. Ecosistema de Agentes de IA

El proyecto requiere el uso combinado y coordinado de dos entornos agénticos que se complementan por su nivel de abstracción:

- **Antigravity (Alto Nivel):** Se encargará de la orquestación arquitectónica, el diseño de los flujos de navegación entre vistas, la definición del estado global de la aplicación (Pinia) y la validación estructural del proyecto.
- **opencode (Bajo Nivel):** Se encargará de la generación de código fuente a nivel de componente individual, la escritura de funciones reactivas, la implementación de estilos CSS utilitarios y la creación de scripts de pruebas unitarias.

1.3. Estrategia de Optimización y Gestión de Tokens

Para evitar el desbordamiento de contexto y optimizar el uso de tokens, el desarrollo se segmentará de forma estrictamente modular. Ningún agente recibirá el PRD completo para generar código; en su lugar, se le proveerá el contexto exclusivo del módulo o componente en el que va a trabajar, acompañado de las reglas globales de estilo y arquitectura.

2. Stack Tecnológico Estándar

- **Framework Principal:** Vue 3 (Versión 3.5+) utilizando obligatoriamente la sintaxis de `<script setup>` y Composition API. Queda estrictamente prohibido el uso de la sintaxis antigua Options API (Vue 2) para asegurar la máxima escalabilidad y el cumplimiento de Clean Code.
- **Gestor de Estado Global:** Pinia (configurado de manera modular y desacoplada).
- **Herramienta de Construcción:** Vite (para garantizar una compilación ultrarrápida y un

Hot Module Replacement instantáneo).

- **Enrutamiento:** Vue Router (para la gestión de SPA - Single Page Application).
- **Estilos:** CSS Utilitario puro (diseñado mediante grillas y flexbox responsivo).

3. Arquitectura del Sistema de Diseño (Atomic Design / AUI)

Para guiar de forma inequívoca a los agentes de opencode y evitar el acoplamiento de código, la carpeta src/components/ se dividirá en tres niveles de abstracción:

3.1. Átomos (Componentes Base / Reutilizables)

Componentes puros que no dependen de ningún almacén de estado y reciben datos únicamente por props.

- **BaseButton.vue:** Botón general configurable (colores primarios naranja/azul). Debe incluir feedback visual para los estados: hover, active, disabled y un estado de loading (spinner dinámico).
- **BaseInput.vue:** Campo de texto reactivo con validación de errores nativa y feedback visual de foco.
- **BaseBadge.vue:** Etiqueta flotante para mostrar contadores (ej. número de ítems en el carrito).

3.2. Moléculas

Combinación de dos o más átomos para formar una unidad funcional.

- **ProductCard.vue:** Tarjeta de producto individual. Muestra imagen, título, precio, valoración por estrellas y un átomo BaseButton con la acción de añadir al carrito.
- **CartItem.vue:** Fila del producto dentro de la vista de la cesta. Incluye controles para aumentar/disminuir la cantidad y eliminar el producto.

3.3. Organismos

Estructuras complejas que gestionan lógica de negocio o interactúan con el estado global.

- **AppNavbar.vue:** Barra superior que contiene el logotipo, la barra de búsqueda general, el selector de idioma, el acceso a la cuenta del usuario y el icono del carrito de compras con su respectivo BaseBadge dinámico conectado a Pinia.
- **ProductGrid.vue:** Rejilla responsiva que renderiza una colección de ProductCard aplicando filtros dinámicos.
- **ChatbotWidget.vue:** Ventana flotante de asistencia al usuario localizada en la esquina inferior derecha.

4. Estructura de Carpetas del Proyecto (src/)

```
src/
├── assets/      # Estilos globales, variables de color e imágenes.
├── components/  # Componentes organizados (Atoms, Molecules, Organisms).
├── views/       # Componentes de página completa (Vistas de Vue Router).
├── store/       # Almacenes globales de Pinia (auth, cart, products).
├── services/    # Capa de red y llamadas a la API (desacoplamiento total).
├── router/      # Configuración de las rutas de la aplicación.
└── App.vue      # Componente raíz con el Layout principal.
```

5. Especificación de Vistas (views/) y Enrutamiento

La aplicación contará con las siguientes páginas gestionadas por vue-router:

- **HomeView.vue ('/')**: Pantalla principal utilitaria. Renderizará la AppNavbar, un banner principal de promociones, y secciones organizadas en bloques destacados: 'Tendencias' y 'Los más vendidos'.
- **CatalogView.vue ('/catalog')**: Vista de resultados de búsqueda. Incluirá un panel lateral izquierdo con filtros categóricos y de rango de precios, y a la derecha el ProductGrid.
- **ProductDetailView.vue ('/product/:id')**: Ficha técnica detallada del artículo. Muestra galería de imágenes, descripción profunda, opiniones de usuarios y acción de compra.
- **CartView.vue ('/cart')**: Desglose detallado de la cesta de la compra. Muestra una lista de CartItem y un panel lateral derecho con el subtotal resumido.
- **CheckoutView.vue ('/checkout')**: Pasarela de pago y envío simulada. Contiene formularios estructurados para dirección de entrega y datos de tarjeta de crédito.

6. Gestión del Estado Común (store/)

Se crearán tres almacenes independientes en Pinia para asegurar la separación de conceptos (Single Responsibility Principle):

- **auth.js**: Almacena el estado de la sesión del usuario (datos del perfil, token JWT simulado y estado de autenticación).
- **products.js**: Gestiona el catálogo de productos cargados, el término de búsqueda actual y los filtros aplicados en el catálogo.
- **cart.js**: Gestiona el array de productos añadidos a la cesta. Contiene las actions para añadir, restar o eliminar productos, y getters para calcular automáticamente el subtotal, los impuestos (IVA del 21%) y el precio total. Sincronizado con localStorage.

7. Desacoplamiento de Servicios y Arquitectura del Chatbot

7.1. Capa de Servicios (services/)

Para evitar que los componentes de Vue realicen llamadas directas de red (lo que rompería el Clean Code y dificultaría el testing), se implementará una capa de servicios intermedios (ej. productService.js, authService.js). Estos archivos encapsularán las peticiones HTTP hacia los endpoints del Back-end.

7.2. Solución Arquitectónica para el Chatbot de Asistencia

- **Corrección de Enfoque:** Se rechaza explícitamente la idea de realizar web scraping directo desde el Front-end debido a problemas de rendimiento, seguridad del lado del cliente y restricciones de CORS.
- **Implementación Correcta:** El componente ChatbotWidget.vue actuará de forma puramente pasiva. Capturará el mensaje del usuario y utilizará el servicio especializado chatbotService.js para enviar el texto mediante un método POST a la API del Back-end. El Back-end procesará los datos y devolverá la respuesta limpia.

8. Requisitos No Funcionales y Control de Calidad (QA)

8.1. Accesibilidad (WCAG 2.1 AA)

- Uso estricto de HTML5 semántico (<header>, <nav>, <main>, <section>, <footer>).
- Inclusión de atributos aria-label en elementos iconográficos o botones interactivos sin texto explícito.
- Garantía de contraste de color adecuado para la paleta elegida (Texto oscuro sobre fondos claros, blanco sobre botones azul/naranja).

8.2. Estrategia de Testing y QA

Los agentes de opencode deberán generar de forma automatizada pruebas unitarias empleando Vitest y Vue Test Utils.

- **Requisito Mínimo Obligatorio:** Cobertura de pruebas unitarias sobre los métodos matemáticos y lógicos del cart.js (comprobar que la suma de productos calcule bien el total y que el IVA se desglose correctamente).
- **Pruebas de Interfaz:** Validar que los botones de la interfaz reaccionen correctamente y emitan los eventos esperados al hacer clic.